周2-3

现阶段需要做的工作

一、Skill 普查：先搞清楚 OpenClaw 有哪些 Skill

1. 具体要干什么？

简单来说，就是调查 OpenClaw 生态中存在的 Skill，了解它们分别能够完成什么任务。

可以从两个来源入手：

- **OpenClaw 官方仓库中的 bundled skills**： 随项目提供的 Skill，适合先研究其结构与调用方式。
- **ClawHub**： 社区发布的 Skill，数量和类型可能更多，适合研究不同依赖及移植难度。

2. 每个 Skill 要调查哪些信息？

建议建立下面这样的表格：

字段	具体记录什么
Skill 名称	原始名称
来源	官方仓库或 ClawHub 链接（后者包括包含 owner/slug、版本或 commit）
功能	用一句话描述
文件结构	只有 SKILL.md，还是包含脚本、资源文件
脚本语言	Python、JS、Shell 等
外部依赖	npm 包、Python 库、命令行工具等
宿主依赖	是否调用 OpenClaw 专有工具或接口
系统能力	文件、网络、进程、浏览器等
初步分类	L1 / L2 / L3 / 待确认
分类依据	为什么判断为该等级
验证状态	静态复核状态：沿用已有评审 / 本次已复核 / 存在争议；原平台测试状态：未测试 / 成功 / 功能失败 / 环境阻塞 / 未完成；鸿蒙验证状态：未验证 / 成功 / 功能失败 / 环境阻塞

建议你们特别注意两个容易混淆的字段：

脚本语言是 Skill 自带的执行代码使用什么语言；宿主依赖是 Skill 是否依靠 OpenClaw 提供的工具和运行机制。

比如某个 Skill 只有 `SKILL.md`，但其中要求智能体调用 `exec` 执行系统命令，它虽然没有自带脚本文件，却仍然存在运行时依赖。

3. 交付物

《OpenClaw Skill 普查表 v0.1》

建议目标是：

- 基于已有 50 个 bundled Skill 和 24 个 ClawHub 样本的静态评审成果，整理普查表，重点复核不少于 10 个样本，最好覆盖环境组主测样本及存在分类争议的样本。保留其余已有判断，并明确复核状态。

二、可移植性分类：判断哪些 Skill 容易移植

刘老师明确提出，要搞清楚哪些 Skill 可以直接移植，哪些需要修改，哪些困难。

因此，你们不能只是给 Skill 随便贴上 L1、L2、L3 标签，而应该制定一套统一的判断标准。

1. 建议采用的初步分类标准

L1：直接或少量修改即可移植

Skill 主要由提示词、流程说明或简单逻辑构成，不依赖难以替换的 OpenClaw 专有能力。

例如：文本格式转换、简单内容总结、固定格式生成。

需要验证：鸿蒙端智能体是否能按原有 Skill 说明执行，而不是只复制 SKILL.md。

L2：需要适配后才能移植

Skill 含有脚本、外部 API 或平台相关逻辑，但其依赖可以通过 ArkTS 改写、接口替换等方式解决。

例如：调用公开 HTTP API 的查询工具、简单 JSON 文件处理。

需要验证：修改后的实现能否保持原有功能及输入输出语义。

L3：存在较大移植障碍

Skill 依赖桌面系统、复杂 Python/Bash 生态、特定进程环境、OpenClaw 专有工具或鸿蒙暂未确认支持的能力。

例如：依赖 Linux 命令行工具链、复杂本地进程控制的 Skill。

需要分析：究竟缺少哪种能力，是否能通过 SkillVM 或其他适配层解决。

这里特别强调：L3 不等于永远不能移植。 它表示当前目标环境下存在明显障碍，可能需要额外运行时、架构调整或者较高改造成本。

对于分类有争议的 Skill，先标记“待确认”，不要为了填满分类表强行判断。

2. 交付物

交付物	具体内容	周日前建议目标
①《Skill 可移植性分类标准 v0.1》	明确 L1/L2/L3 的定义、判断条件、分类流程及典型依赖障碍	完成初稿
②《Skill 可移植性分类结果表 v0.1》	对普查组提供的真实 Skill 逐个分析，记录分类结果和依据	初步分类 10～15 个

三、原平台基线测试：证明 Skill 原来是怎么工作的

这一步非常重要，而且容易被忽略。

移植之前必须知道原来的 Skill 实际能做什么，否则移植后无法判断功能是否正确。

举个例子。

假设某个 Skill 的功能是将输入的 JSON 数据转换为 CSV。

原平台输入：

```
[
  {"name": "Alice", "age": 20},
  {"name": "Bob", "age": 22}
]
```

预期输出：

```
name,age
Alice,20
```

```
Bob,22
```

你们需要在 OpenClaw 原平台上运行该 Skill，确认它的实际行为，而不只是根据说明文档猜测。

1. 具体怎么测试?

每个样本建议执行以下流程：

1. 阅读 `SKILL.md`，确认触发条件和预期功能。
2. 检查脚本和依赖，确认运行前需要配置什么。
3. 在 OpenClaw 中实际调用 Skill。
4. 记录输入、输出、日志、错误信息。
5. 保存至少一个正常案例，条件允许再测试一个异常案例。

2. 交付物

交付物	具体内容	周日前建议目标
①《OpenClaw Skill 基线测试记录表》	记录每个 Skill 的环境、配置、输入、预期输出、实际输出、运行状态	每人完成 1 个主测 Skill 的完整基线记录；第二个作为备选或有余力时扩展。遇到阻塞须记录具体步骤与证据，不要求所有样本跑通。
②《基线测试证据包》	保存调用日志、截图、输出文件、必要的配置说明	每个已测 Skill 至少一组证据
③《Skill 运行问题与实际依赖清单》	汇总运行中发现的依赖、报错、配置问题和阻塞原因	完成一份汇总表

注：

最重要的是《基线测试记录表》，建议采用统一格式

字段	应填写的内容
Skill 名称与来源	名称、bundled/ClawHub、来源链接及版本
测试环境	操作系统、OpenClaw 版本、Node.js 版本、模型提供方
安装与配置	安装命令、依赖、必要配置（密钥不能明文记录）
调用方法	使用的提示词、触发方式或命令
测试输入	实际输入内容

字段	应填写的内容
预期行为	根据 Skill 说明事先确定的功能
实际输出	真实执行结果或输出文件
执行证据	日志、截图、文件路径
测试状态	成功 / 功能失败 / 环境阻塞 / 未完成
实际依赖	运行时观察到的工具、脚本、网络或文件操作
后续问题	需要分类组或甲组进一步确认的问题
测试范围	完整流程，还是某个功能子集
skill使用确认	已确认 / 未确认，并附加载、工具调用或脚本执行证据

特别注意：模型给出了正确答案，不一定代表它真的调用了目标 Skill。

例如你让 OpenClaw 总结一段文本，即使没有加载任何特定 Skill，模型也可能完成任务。

因此需要尽量确认 Skill 确实被加载、相关指令被使用，或者目标脚本、工具确实执行了。无法确认时，应如实标记。如果只在终端直接运行附带脚本，可以记为“脚本级测试成功”，但不能据此宣称“OpenClaw 调用链已验证”。这一点适用于所有环境组成员。

分工

结合我们乙组前期的工作基础以及比较紧迫的工期，这周我们先继续沿用第一周的两个 4 人小组，分别推进 Skill 静态分析和动态实验。

原来阅读源码的4人：（负责普查和分类）

人员	任务	交付物	姓名
S1	复核已有50项 bundled 调查的准确性，补充遗漏的功能与依赖证据，形成可阅读的整理结果	bundled Skill 普查表，注意保留本次复核范围及修改记录	李萌
S2	复核已有社区24个评审样本的版本、功能与依赖，说明选样覆盖范围和局限	ClawHub Skill 普查表，注意保留本次复核范围及修改记录	林思颖

人员	任务	交付物	姓名
S3	基于已有判断制定统一 L1/L2/L3 标准，复核主测样本，并吸收运行结果	分类标准 v0.1 + 分类结果表	李元媛
S4	简短交接材料、解释数据与脚本，集中解答少量疑问	说明文件用途、复现方法和已知不足；不再承担新增普查	祝一菲

注：

建议S1与S2同学均

- 对负责范围做基础完整性检查。
- 各选约 5 个样本进行源码重点复核。
- 优先覆盖环境组主测样本，其次处理证据缺失和争议条目。
- 未重新核对的内容保留原结论，注明“沿用已有静态评审”。

原来配置环境的4人：

每人从已有文档中自行选择一个主测样本、一个备选样本进行基线测试

人员	验证重点	选择要求	姓名
E1	指令加载与执行	选择纯描述或低依赖 Skill，重点确认目标 Skill 确实被加载和使用	孙冉
E2	外部服务调用	选择以 HTTP/API 为核心的 Skill，重点记录认证、请求、响应和错误处理	娄懿
E3	本地脚本执行	选择以本地文件或数据处理为核心的 Python/JS 等脚本型 Skill，重点记录运行时、依赖和输出文件	张庆涵
E4	宿主工具或进程协作	选择依赖 OpenClaw 工具、会话或进程管理的 Skill，重点验证调用链和阻塞位置	邱莉扉

注：

1. E4 如果遇到无法运行的复杂 Skill，不要求一定跑通，但必须说明阻塞发生在哪一步、证据是什么、属于环境配置问题还是 Skill 本身的执行问题。
2. 环境组这些分工不预设 L1/L2/L3。